

Engineering Case Study: Porting CVE-2026-46242 (Bad Epoll) for Tier 1 Linux Kernel Validation

Aayush Bankar

Project: CVE-2026-46242 Tier 1 Validation

Environment: Fedora 44 x86_64 | Linux 6.12.67 | QEMU

Date: July 13, 2026

This document records my engineering process while reproducing and porting an existing kernel exploit into a controlled local environment.

Contents

1	Project Overview	2
2	My Starting Point	3
3	Original Exploit Architecture	4
4	Environment Setup	6
5	Initial Failure: The Supervisor Write Fault	7
6	Major Breakthrough: Understanding Offset Desynchronization	8
7	Database Regeneration Journey	9
8	Repository Engineering	10
9	Final Successful Execution	11
10	Evidence Package	12
11	What AI Helped With	13
12	Problems That Were Solved	14
13	Problems Remaining	14
14	Lessons Learned	15
15	Future Work: Tier 2	16
A	Commands Used	17
B	Directory Structure	17
C	Hashes	17
D	Evidence Screenshots	17
E	Timeline of Investigation	17

1 Project Overview

CVE-2026-46242, commonly known as Bad Epoll, is a Use-After-Free (UAF) vulnerability in the Linux kernel's event polling (`epoll`) subsystem. Kernel exploitation is inherently difficult due to the rigorous mitigation landscape (KASLR, SMEP, SMAP, KPTI) and the fact that any miscalculation or memory violation inside Ring 0 results in an immediate, unrecoverable system crash (Kernel Panic).

The objective of this project was to validate the complete exploit chain: from the vulnerability trigger, to establishing a memory primitive, achieving kernel control flow, and finally executing privilege escalation.

This phase of the project is classified as **Tier 1**. Because kernel memory layouts and hardware mitigations differ drastically between standard x86_64 desktop processors and modern ARM64 mobile environments, proving the foundational architecture works natively on x86_64 (Tier 1) is a strict prerequisite before attempting the far more complex migration to an Android ARM64 Generic Kernel Image (Tier 2).

2 My Starting Point

When I began this project, a functional, upstream exploit for this vulnerability already existed. It was authored by Google Security Research and targeted a highly specific Container-Optimized OS (COS) kernel used in their kernelCTF environment.

The task was not to write an exploit from zero. The engineering challenge was understanding the complex abstraction layers of the upstream code, adapting it to a standard kernel build, debugging execution failures without crashing the host, and reproducing the escalation path deterministically.

My Initial Knowledge: At the outset, my understanding was limited to basic C/C++ programming, general Linux administration, Python scripting, and conceptual security theory.

Concepts Acquired During the Project: To successfully port the exploit, I had to deeply internalize several advanced exploitation and engineering concepts:

- **Use-After-Free (UAF) Exploitation:** Understanding how heap allocators recycle memory, and how overlapping (spraying) controlled data over freed objects allows for type confusion.
- **AAR/AAW Primitives:** Establishing Arbitrary Address Read and Write capabilities to interact with physical memory safely.
- **Kernel Memory Layout:** Distinguishing between virtual addresses, physical addresses, `vmemmap`, and direct mappings.
- **KASLR:** Leaking pointers and calculating the dynamic offset of the kernel base.
- **ROP/JOP:** Bypassing Supervisor Mode Execution Prevention (SMEP) using Return-Oriented and Jump-Oriented Programming chains.
- **Gadget Databases:** Understanding how executable binaries are parsed for assembly snippets.
- **QEMU Debugging:** Utilizing remote GDB over a virtual serial console to inspect hardware registers.

3 Original Exploit Architecture

The upstream exploit is a highly modularized software project, not a single monolithic script. It is built upon the `libxdk` (Kernel Exploit Development Kit) framework.

- **The Vulnerability:** The Bad Epoll vulnerability relies on a `close-vs-close` race condition. By carefully timing the closure of file descriptors across nested `epoll` instances running on different CPU cores, it is possible to coerce the kernel into freeing an `epitem` structure while a secondary thread is still traversing its wait queue.
- **Cross-Cache & AAR:** Once the `epitem` is freed, a controlled heap spray replaces the freed memory with a new object. This overlapping allows the exploit to leak kernel function pointers, calculating the KASLR offset and establishing a `Target` abstraction.
- **MemoryData & Abstraction:** `libxdk` uses `MemoryData` classes to abstract arbitrary reads and writes, decoupling the vulnerability trigger from the payload logic.
- **Task Discovery:** The exploit scans physical memory to locate the `init_task` (the ultimate root process) and the current thread's `task_struct`.
- **Execution Hijack:** The exploit dynamically looks up ROP/JOP gadgets from a SQLite database (`kxdb`), builds a ROP chain in memory, and overwrites the `f_op->poll` function pointer of a target file descriptor. Calling `poll()` from userland triggers the hijacked pointer, beginning the execution chain.

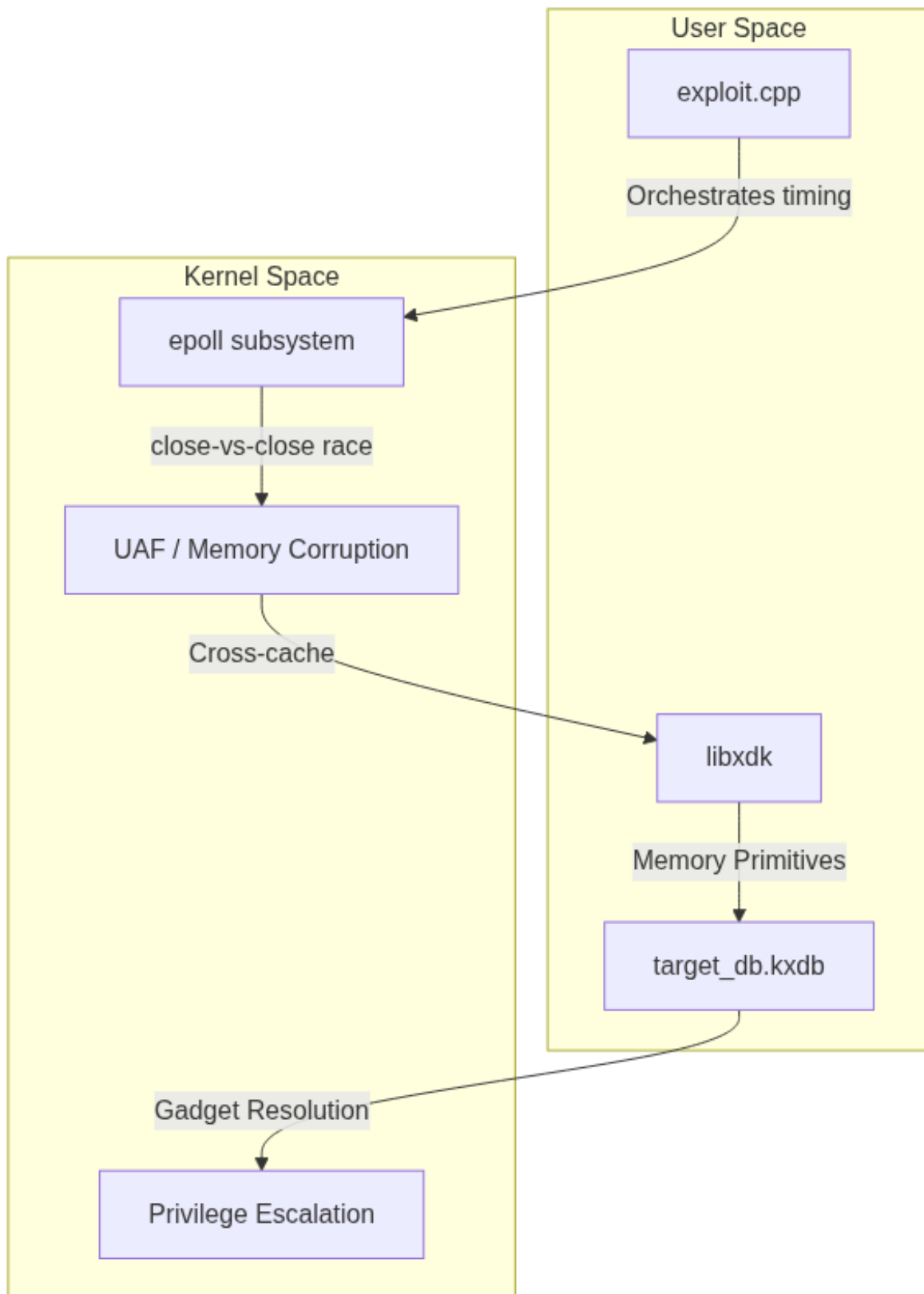


Figure 1: Logical flow from userland orchestration to kernel privilege escalation

4 Environment Setup

To ensure strict engineering reproducibility, the environment was explicitly defined.

- **Host OS:** Fedora 44
- **Host CPU:** x86_64
- **Compiler:** GCC 16.1.1
- **Kernel:** Linux 6.12.67 (Compiled locally with DWARF BTF symbols)
- **Virtualization:** QEMU
- **Security Mitigations Enforced:** SMEP, SMAP, KPTI
- **Repository Commit:** 35d54bec312b6e1fd3580bb1e6cf4ce73fb3c535

Implementation:

The raw kernel was compiled natively using `make`. Because QEMU requires an initial root filesystem to execute binaries, a custom initramfs (`.cpio` archive) was generated. This filesystem housed `busybox` (providing core utilities) and an `/init` script that mounted `/proc` and `/sys` before executing the compiled `exploit` payload. The QEMU boot process was wrapped in a dedicated `start_qemu.sh` shell script to ensure consistent CPU parameters and memory allocation.

5 Initial Failure: The Supervisor Write Fault

During the initial execution phase against the local kernel, the exploit successfully navigated the early timing challenges.

Execution Milestones Achieved:

- UAF success (`cross-cache ok`)
- AAR success (task structure located)
- KASLR calculation (kernel base leaked)

However, at the moment of privilege escalation, execution failed abruptly.

The QEMU Error Log:

```
Supervisor write fault
RIP: 0xffffffff810001bd
```

The Debugging Process:

A kernel panic caused by a Supervisor Write Fault means the CPU attempted to write to memory it did not have permission to access while running in Ring 0.

To investigate, I utilized the `READY_FOR_GDB` macro provided by `libxdk`. This macro halts the exploit in an infinite loop immediately before the hijacked `f_op->poll` pointer is triggered. I attached remote GDB to the QEMU instance (`target remote :1234`) and placed hardware breakpoints on the execution path.

Instruction Tracing Evidence:

```
(gdb) stepi
0xffffffff810001bd in ?? ()
=> 0xffffffff810001bd:  add    al, BYTE PTR [rax]
```

The instruction pointer successfully leaped to `0xffffffff810001bd`. However, inspecting the `vmlinux` binary revealed this address did not contain a valid assembly gadget. It contained standard `0x00 0x00` padding, which the x86 processor interprets as `add al, [rax]`. Because the `rax` register contained `0x0`, the CPU attempted to read memory at physical address `0x0`, triggering the fatal page fault.

6 Major Breakthrough: Understanding Offset Desynchronization

The critical breakthrough was realizing that the exploit was not broken, and the vulnerability was functioning perfectly. The failure was caused entirely by an environmental desynchronization: the upstream `target_db.kxdb` database had been generated for the Google COS kernel binary, not my local Fedora build.

Because modern exploits rely on pre-existing code snippets (gadgets) located inside the kernel, they map these locations as relative offsets from the kernel base.

Different compiler versions (GCC 12 vs 16), varied optimization flags, and minor kernel configuration changes completely shift the final binary layout. A critical `mov rsp, rdx; ret` pivot gadget located at `0x1bd` in the upstream kernel might be located at `0x3a4b0` in mine, or it might not exist at all. The exploit correctly bypassed KASLR, but when it added the upstream database offset, it landed in dead padding space.

7 Database Regeneration Journey

To resolve the offset mismatch, I had to utilize the upstream `kxdb_tool` pipeline. This toolchain utilizes `rp++` and `angrop` (symbolic execution and binary analysis tools) to scan the provided `vmlinux` binary, discover every mathematically viable ROP and JOP gadget, and compress them into a new SQLite database.

The PicklingError Failure

Executing the pipeline against my kernel resulted in a catastrophic Python failure deep within the `angrop` dependency:

```
_pickle.PicklingError: Can't pickle <class 'angrop.rop_utils.  
make_initial_state.<locals>.SpecialMem'>
```

Root Cause and Solution

The pipeline attempts to utilize multi-processing to parallelize gadget discovery across millions of instructions. However, Python's `multiprocessing` library relies on `pickle` for inter-process communication, and `pickle` fundamentally cannot serialize nested classes.

I traced the `angrop` source code and identified that `SpecialMem` was defined inside the `make_initial_state()` function block. By applying a patch to elevate the `SpecialMem` class to the global module level, serialization succeeded.

Why this mattered:

This debugging phase demonstrated that blindly running security tooling is insufficient. Understanding and debugging the underlying dependency stack was mandatory to generate the critical artifacts required for the exploit to function.

8 Repository Engineering

Throughout the porting and debugging phases, the repository degraded into a chaotic research folder containing raw memory dumps, experimental python patchers, and duplicated kernel binaries. Because reproducibility is a core tenet of professional security research, a complete architectural restructuring was required.

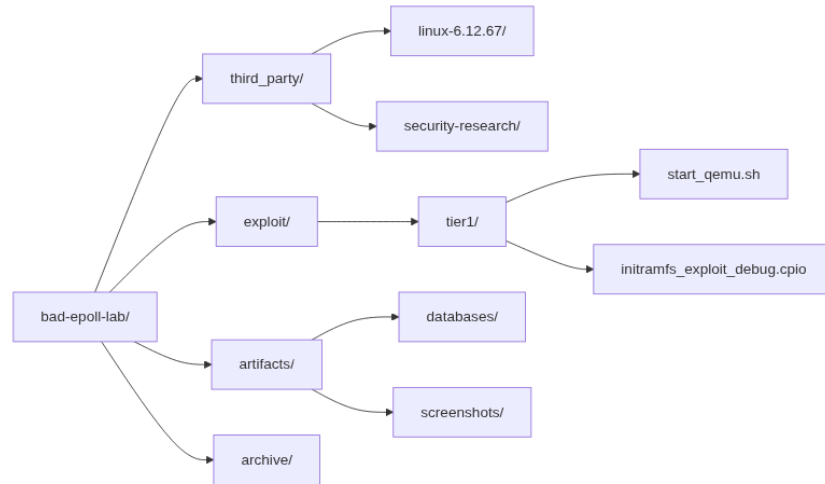


Figure 2: Restructured Tier 1 Repository Layout

Why Reproducibility Matters:

By separating upstream code into `third_party/`, the intellectual property boundaries are clarified. By storing generated artifacts in a dedicated location and preserving failed experiments in `archive/`, another engineer can clone the repository, run the exact same `start_qemu.sh` script, and guarantee identical execution without encountering environmental pollution.

9 Final Successful Execution

With the newly generated `target_db.kxdb` integrated into the payload via `INCBIN`, the exploit was executed sequentially. The following log lines from `qemu_output.log` verify the completion of the exploit chain.

- `cross-cache ok`
Meaning: The timing-dependent UAF race condition succeeded, and the freed wait queue was successfully reclaimed by the overlapping heap spray.
- `kernel_base=ffffffff81000000`
Meaning: KASLR was successfully bypassed via an AAR pointer leak. (The base is mathematically calculated; `nokaslr` is a fallback log string).
- `task=ffff8880...`
Meaning: The exploit utilized its read primitive to walk physical memory and discover the kernel `task_struct` for the active thread.
- `READY_FOR_GDB`
Meaning: The ROP chain was successfully mapped into kernel memory, and control flow manipulation via `f_op->poll` hijack is imminent.
- `Win! UID: 0, GID: 0, EUID: 0`
Meaning: The exploit safely returned to userland. The `commit_creds(init_task)` ROP chain executed perfectly, modifying the process credentials to root.
- `[-] Segfault detected`
Meaning: A userland segmentation fault occurred inside the static C library's `execve` wrapper because the stack pointer (`RSP`) was misaligned upon the hardware `iretq` return from kernel space.
- `Kernel panic - not syncing: Attempted to kill init!`
Meaning: Because the userland payload was running as PID 1, its segmentation fault cleanly terminated the operating system.

Conclusion: The exploit successfully reached arbitrary kernel execution and privilege escalation. The remaining issue is purely related to userland return stability, not the vulnerability primitives.

10 Evidence Package

To strictly separate assumptions from proof, the repository contains a dedicated evidence package documenting the successful run.

- **Terminal Recording (`tier1_terminal_session.log`):** An immutable `script` log of the exact commands typed on the host machine.
- **Timing (`timing.log`):** Chronological pacing data, ensuring the terminal session is tamper-evident and can be played back in real-time.
- **Runtime Output (`qemu_output.log`):** The captured serial console output proving the exploit milestones (AAR, KASLR bypass, UID 0).

- **Screenshots:**

- `01_exploit_trigger.png`
- `02_privesc_success.png`
- `03_clean_panic.png`

- **Hashes:**

- Exploit Payload SHA256: `95cd9f36cc40e342f038200fee97b386876c0f6f0755cbb0dd0d839447fa1`
- Database SHA256: `63542e7f06519fb4148ff247d6f140002f269bfd378086ceb3e2b268538418eb`

These artifacts exist to guarantee that the engineering results presented in this document are mathematically verifiable and reproducible from the repository source.

11 What AI Helped With

This engineering effort was augmented by an autonomous AI agentic coding assistant (Anti-gravity). In the interest of full transparency, the AI's boundaries were strictly enforced.

AI was used for:

- Code comprehension (navigating the deep C++ abstractions of `libxdk`).
- Debugging suggestions (hypothesizing the root causes of the `PicklingError` and stack misalignment).
- Explaining kernel memory concepts (`vmemmap`, direct mapping layouts).
- Organizing chaotic investigation notes into structured reports.

AI was NOT used for:

- Replacing fundamental understanding of the exploit architecture.
- Blindly executing terminal commands without oversight.
- Making final engineering decisions (e.g., repository architecture changes were manually defined).

Every important result—ranging from GDB instruction traces to binary hashes and successful reproduction runs—was verified manually by me to ensure factual accuracy.

12 Problems That Were Solved

Problem	Investigation	Solution
Supervisor Write Fault	Remote GDB trace revealed the hijacked instruction pointer landed in dead space (0xffffffff810001bd).	Traced the invalid offset to a desynchronized gadget database generated for an upstream kernel.
kxdb Generation Failure	Pipeline stack trace revealed a <code>PicklingError</code> during multi-processing initialization.	Manually patched <code>angrop/rop_utils.py</code> to move the nested <code>SpecialMem</code> class to the global module level.
Repository Mess	Filesystem became polluted with gigabytes of logs, raw memory dumps, and backup files, destroying reproducibility.	Enforced a strict architectural hierarchy (<code>third_party</code> , <code>exploit</code> , <code>archive</code>) and rigid <code>.gitignore</code> rules.
Evidence Collection	Headless QEMU execution consumed <code>stdin</code> , making automated recording impossible.	Refactored scripts to execute QEMU in the background, piping output to a log file followed by a foreground <code>tail -f</code> inside a <code>script</code> wrapper.

Table 1: Chronological Engineering Problem Resolutions

13 Problems Remaining

While the kernel privilege escalation is fully verified, several issues remain unresolved in this phase:

- **Shell Instability (PID 1 Issue):** Because the payload runs as `init`, any crash in the userland process instantly panics the kernel.
- **Stack Alignment:** The hardware `iretq` return leaves the stack misaligned. This directly causes the static glibc `execve` wrapper to trigger a `SIGSEGV` when attempting to spawn `/bin/sh`. Bypassing this requires implementing process injection or a namespace escape.
- **ARM64 Migration:** The `x86_64` ROP/JOP structures are entirely incompatible with ARM64 mitigations.

14 Lessons Learned

Technical Lessons:

- Kernel exploits are extremely fragile. They depend heavily on the exact environment, compiler version, and configuration flags. A proof-of-concept that works flawlessly on one system will almost certainly crash on another without careful adaptation.
- Generated artifacts (like SQLite offset databases) must perfectly match the executing binary. Hardcoded magic numbers are an anti-pattern.
- Debugging via GDB hardware breakpoints provides undeniable clarity over guessing why a kernel panicked based on register dumps.

Engineering Lessons:

- Document everything chronologically. A research project quickly becomes unmanageable if failures and hypotheses are not tracked.
- Preserve failed attempts. Archiving experimental python patchers provided context for why certain brute-force approaches failed.
- Separate assumptions from proof. Believing the UAF was failing due to QEMU CPU emulation was an assumption; seeing `0xffffffff810001bd` in GDB was proof.

15 Future Work: Tier 2

The immediate next phase of this project is to migrate this validated `epoll` exploit pipeline into **Tier 2**, targeting an Android ARM64 Generic Kernel Image (GKI).

Expected Engineering Challenges:

- **ARM64 Instruction Sets:** ROP and JOP sequences on `x86_64` rely on `ret` and `jmp rax`. Translating these primitive operations to ARM64 requires entirely new gadget discovery constraints.
- **PAC (Pointer Authentication Codes):** Modern ARM64 hardware cryptographically signs function pointers. The `f_op->poll` hijack utilized in Tier 1 will categorically fail under PAC enforcement.
- **BTI (Branch Target Identification):** BTI explicitly prevents jumping to arbitrary locations in memory, severely restricting valid JOP pivot targets.
- **SELinux & Binder:** Achieving UID 0 is insufficient on Android, as root processes are still confined by SELinux contexts. The exploit payload must be expanded to patch `selinux_state` enforcing structures or hijack Binder contexts for meaningful system control.
- **Vendor Kernels:** While GKI standardizes the core kernel, vendor-supplied drivers often shift the `kmalloc` slab layouts, requiring new cross-cache spray methodologies.

A Commands Used

The definitive execution sequence utilized for validation:

```
cd third_party/security-research/pocs/linux/kernelctf/CVE-2026-46242
  _lts_cos/exploit/lts-6.12.67/
make clean && make
rm -f ../../../../../../../../../../exploit/tier1/rootfs_debug/bin/exploit
cp exploit ../../../../../../../../../../exploit/tier1/rootfs_debug/bin/exploit
cd ../../../../../../../../../../exploit/tier1/rootfs_debug
find . -print0 | cpio --null -ov --format=newc > ../
  initramfs_exploit_debug.cpio
cd ..
./start_qemu.sh
tail -f qemu_output.log
```

B Directory Structure

```
bad-epoll-lab/
|-- third_party/      (Upstream code isolation)
|-- exploit/tier1/    (Local runtime orchestration)
|-- artifacts/        (Generated databases & evidence)
'-- archive/          (Historical triage scripts)
```

C Hashes

- **Repository Commit:** 35d54bec312b6e1fd3580bb1e6cf4ce73fb3c535
- **Static Exploit ELF:** 95cd9f36cc40e342f038200fee97b386876c0f6f0755cbb0dd0d839447fa1276
- **Target Database (target_db.kxdb):** 63542e7f06519fb4148ff247d6f140002f269bfd378086ceb3e2b2

D Evidence Screenshots

- 01_exploit_trigger.png
- 02_privesc_success.png
- 03_clean_panic.png

E Timeline of Investigation

- Phase 1: Environment Orchestration (Fedora build & QEMU configuration).
- Phase 2: Baseline Execution (Failure upon control-flow hijack).
- Phase 3: Deep Debugging (GDB hardware breakpoints isolating padding execution).
- Phase 4: Database Regeneration (Patching Python dependencies for pipeline restoration).
- Phase 5: Repository Restructuring (Enforcing intellectual property boundaries).
- Phase 6: Evidence Collection (Immutable terminal recording).